

Problem Sheet 2

Questions to answer: 2.1, 2.2, 2.4 (a,b), 2.7, 3.1, 3.2, 3.4.

2.1: Write a program that finds the integer part of $\sqrt{x}$ using binary search, and test it by initially setting x to 200 000 000. Compile your program into Keiko code, and work out the purpose of each instruction.

Here is the program for the binary search version of the square root algorithm in Pascal.

```
(* prob_sheet_2/bs.p *)
begin
  x := 200000000;
  low := 0;
  high := x;

  if x < 0 then
    print -1
  else
    if x < 1 then
      high := 1
    end;

    while (high - low) > 1 do
      mid := (low + high) div 2;

      if mid > 0 then
        if mid > x div mid then
          high := mid
        else
          low := mid
        end
      else
        low := mid
      end
    end;

    print (low + high) div 2;
    newline
  end
end.
```

And here is the Keiko code generated from this:

```

MODULE Main 0 0
IMPORT Lib 0
ENDHDR

FUNC MAIN 0
! x := 200000000;
CONST 200000000
STGW _x
! low := 0;
CONST 0
STGW _low
! high := x;
LDGW _x
STGW _high
! if x < 0 then
LDGW _x
CONST 0
JLT L1
JUMP L2
LABEL L1
CONST 1
UMINUS
CONST 0
GLOBAL lib.print
PCALL 1
JUMP L3
LABEL L2
! if x < 1 then
LDGW _x
CONST 1
JLT L4
JUMP L5
LABEL L4
! high := 1
CONST 1
STGW _high
JUMP L6
LABEL L5
LABEL L6
JUMP L8
LABEL L7
! mid := (low + high) div 2;
LDGW _low
LDGW _high
PLUS
CONST 2

```

```

DIV
STGW _mid
! if mid > 0 then
LDGW _mid
CONST 0
JGT L10
JUMP L11
LABEL L10
! if mid > x div mid then
LDGW _mid
LDGW _x
LDGW _mid
DIV
JGT L13
JUMP L14
LABEL L13
! high := mid
LDGW _mid
STGW _high
JUMP L15
LABEL L14
! low := mid
LDGW _mid
STGW _low
LABEL L15
JUMP L12
LABEL L11
! low := mid
LDGW _mid
STGW _low
LABEL L12
LABEL L8
! while (high - low) > 1 do
LDGW _high
LDGW _low
MINUS
CONST 1
JGT L7
JUMP L9
LABEL L9
! print (low + high) div 2;
LDGW _low
LDGW _high
PLUS
CONST 2
DIV

```

```

CONST 0
GLOBAL lib.print
PCALL 1
CONST 0
GLOBAL lib.newline
PCALL 0
LABEL L3
RETURN
END

GLOVAR _low 4
GLOVAR _mid 4
GLOVAR _x 4
GLOVAR _high 4

```

Conditionals are kept the same in *.k files, so whenever I used an if statement, you can see it as exactly that, but without the `then` keyword, but instead with a `|` key letter at the start of the statement.

You can tell that some of the instructions, specifically the JUMP instructions are there specifically for branching (aka loops).

2.2: Some machines have an expression stack implemented in hardware, but with a finite limit on its depth. For these machines, it is important to generate postfix code that makes the maximum stack depth reached during execution as small as possible.

1. Let the SWAP instruction be defined so that it swaps the two top elements of the stack. Show how to use this instruction to evaluate the expression $1/(1+x)$ without ever having more than two items on the stack.

1. PUSH 1	[1]
2. PUSH x	[1, x]
3. ADD	[1+x]
4. PUSH 1	[1+x, 1]
5. SWAP	[1, 1+x]
6. DIV	[1/(1+x)]

2. Prove that if expression e_1 (containing variables, constants and unary and binary operators) can be evaluated in depth d_1 , and e_2 can be evaluated in depth d_2 , then $\text{Binop}(w, e_1, e_2)$ can be evaluated in depth $\min(\max(d_1, (d_2 + 1)), (\max((d_1 + 1), d_2)))$.

Write a function $\text{cost} : \text{expr} \rightarrow \text{int}$ that calculates the stack depth that is needed to evaluate an expression by this method. Show that if e has fewer than 2^N operands, then $\text{cost } e \leq N$.

```
type expr =
  | Const of int
  | Var of string
  | Binop of string * expr * expr

let rec cost e =
  match e with
  | Const _ -> 1
  | Var _ -> 1
  | Binop (_, e1, e2) ->
    let d1 = cost e1 in
    let d2 = cost e2 in
    min (max d1 (d2 + 1)) (max (d1 + 1) d2)
```

Suppose you take e and split it in half, with $e = e_1 e_2$. Running $\text{Binop}(w, e_1, e_2)$ and calculating the cost will result in the depth being $\min(\max(\frac{d_1}{2}, \frac{d_2+1}{2}), \max(\frac{d_1+1}{2}, \frac{d_2}{2}))$. Doing this recursively gives us a recursively, you can see that diving expression in half will result in $e < N$ if the number of operands are less than 2^N .

3. Write an expression compiler $\text{gen_expr} : \text{expr} \rightarrow \text{code}$ that generates the code that evaluates an expression e within stack depth $\text{cost } e$. [Hint: use cost in your definition.]

```
type expr =
  | Const of int
  | Var of string
  | Binop of string * expr * expr (* binary operation like +, -, etc. *)

type code =
  | PushConst of int
  | LoadVar of string
  | BinaryOp of string
  | Swap (* hypothetical instruction to manage stack depth *)

let rec gen_expr (e : expr) : code list =
  match e with
  | Const n -> [PushConst n]
  | Var x -> [LoadVar x]
  | Binop (op, e1, e2) ->
    let d1 = cost e1 in
    let d2 = cost e2 in
    let l1 = gen_expr e1 in
    let l2 = gen_expr e2 in
    l1 @ l2 @ [BinaryOp op]
```

```

let d2 = cost e2 in
if d1 >= d2 then
  (* Evaluate e1 first, then e2, keeping depth within bounds *)
  gen_expr e1 @ gen_expr e2 @ [BinaryOp op]
else
  (* Evaluate e2 first, then e1, with a swap to maintain stack order
  *)
  gen_expr e2 @ gen_expr e1 @ [Swap; BinaryOp op]

```

2.4: Programs commonly contain nested `if` statements, so that either the then part or (more commonly) the else part of an `if` statement is another `if` statement. (The latter possibility can be abbreviated using the `elsif` syntax that was the subject of problem 1.6.)

1. Show the code that is produced for such nested statements by the naive translation scheme that was described in the lectures and used in Lab 1. Point out where this code is untidy and where it is significantly inefficient.

If you look into the `case.p` file, you will find the following program:

```

while i < 10 do
  case i of
    1, 3, 5:
      i := i + 1;
      i := i + 2
    | 2, 6:
      i := i - 1;
    | 8:
      i := i + 2;
  else
    i := i + 1
  end;
  print i; newline
end

```

You can see that these naive nested statements are inefficient, usually due to the fact that there are unnecessary extra end jumps, specifically when compiled into `keiko` code.

2. Suggest rules that could be used in a peephole optimiser to improve the code from part (a), tidying it up and ameliorating any inefficiencies.

To improve the code generated for nested if statements using a peephole optimiser, we can apply several rules. These rules aim to eliminate redundant instructions, optimise jumps, and streamline control flow.

We can do the following:

- Remove redundant jumps. If a JUMP is followed immediately by a LABEL, then we can get rid of that initial JUMP statement.
- If two LABEL statements are together, we can equate and merge them into one.
- If a JUMPC is followed by a JUMP and the LABEL of the JUMPC is the same as the next LABEL, invert the condition and remove the JUMP.
- If a LABEL is not referenced by any JUMP or JUMPC, remove it.
- Combine LOAD and STORE operations with addressing into single instructions where possible.

2.7: Some programming languages provide conditional expressions such as

```
`if i >= 0 then a[i] else 0`
```

which evaluates to `a[i]` if `i >= 0`, and otherwise evaluates to zero without attempting to access the array element `a[i]`.

1. Suggest an abstract syntax for this construct, and suggest a way of incorporating the construct into an `ocaml yacc` parser for a simple programming language so as to provide maximum flexibility without introducing ambiguity. Make sure that an expression like `if x then y else p+q` has `p+q` as a sub expression.

Answer: Here is the abstract syntax for this construct:

```
type expr =  
  | Const of int  
  | Var of string  
  | ArrayAccess of string * expr      (* Represents a[i] *)  
  | Binop of string * expr * expr    (* Binary operations *)  
  | Conditional of expr * expr * expr (* if cond then expr1 else expr2 *)
```

And here, we can give a proper definition to the `expr` type:

```

expr:
| IF expr THEN expr ELSE expr { Conditional($2, $4, $6) }
| expr '+' expr               { Binop("+", $1, $3) }
| expr '-' expr               { Binop("-", $1, $3) }
| expr '*' expr               { Binop("*", $1, $3) }
| '(' expr ')'                { $2 }
| INT                          { Const($1) }
| VAR                          { Var($1) }
| VAR '[' expr ']'            { ArrayAccess($1, $3) }

```

In a compiler for the language, postfix code for expressions is generated by a function:

$\text{gen_expr} : \text{expr} \rightarrow \text{code}.$

Control structures are translated using a function:

$\text{gen_cond} : \text{expr} \rightarrow \text{codelab} \rightarrow \text{codelab} \rightarrow \text{code},$

defined so that `gen_cond e tlab flab` generates code that jumps to label `tlab` if expression `e` has boolean value **true**, and to label `flab` if it has value **false**.

2. Show how to enhance `gen_expr` and `gen_cond` to deal appropriately with conditional expressions.

```

let rec gen_expr e =
  match e with
  | Const n -> [PushConst n]
  | Var x -> [LoadVar x]
  | ArrayAccess (a, i) -> gen_expr i @ [LoadArray a]
  | Binop (op, e1, e2) -> gen_expr e1 @ gen_expr e2 @ [BinaryOp op]
  | Conditional (cond, expr1, expr2) ->
    let true_label = new_label () in
    let end_label = new_label () in
    gen_cond cond true_label end_label
    @ [Label true_label]
    @ gen_expr expr1
    @ [Jump end_label; Label end_label]
    @ gen_expr expr2

```

And to deal with conditional branches, we may use the following:

```

let rec gen_cond cond tlab flab =
  match cond with
  | Binop ("<", e1, e2) -> gen_expr e1 @ gen_expr e2 @ [Jumpc ("LT", tlab);
    Jump flab]

```

```

| Binop (">=", e1, e2) -> gen_expr e1 @ gen_expr e2 @ [Jumpc ("GE", tlab);
Jump flab]
| Binop ("==", e1, e2) -> gen_expr e1 @ gen_expr e2 @ [Jumpc ("EQ", tlab);
Jump flab]
| Binop ("!=", e1, e2) -> gen_expr e1 @ gen_expr e2 @ [Jumpc ("NE", tlab);
Jump flab]
| _ -> failwith "Unsupported condition"

```

It is suggested that short-circuit boolean and could be translated by getting the parser to treat e_1 and e_2 as an abbreviation for the conditional expression

`if e1 then e2 else false`, expanding the abbreviation in creating the abstract syntax tree.

3. Show the code that would be generated for the statement

```
if (i >= 0) and (a[i] > x) then i := i+1 end
```

according to your translation, assuming both i and x are global integer variables, and a is a global array of integers. Omit array bound checks. If the resulting code is longer or slower than that produced by translating the `and` operator directly, suggest rules for post-processing the code so that it is equally good.

```

; Evaluate `i >= 0`
LDGW i          ; Load global variable i
CONST 0         ; Load constant 0
JUMPC Lt, flab  ; Jump to flab if i < 0 (i >= 0 is false)

; Evaluate `a[i] > x`
LDGW i          ; Load index i
GLOBAL a        ; Load base address of array a
BOUNDS          ; (Optional) Check array bounds
LOADW           ; Load a[i]
LDGW x          ; Load global variable x
JUMPC Leq, flab ; Jump to flab if a[i] <= x (a[i] > x is false)

; If both conditions are true, execute `i := i + 1`
LABEL tlab      ; Target for true branch
LDGW i          ; Load global variable i
CONST 1         ; Load constant 1
BINOP Plus      ; Compute i + 1
STGW i          ; Store result back to i

```

flab:

```
; End of if statement
```

3.1: Assume the following declarations.

```
type dogptr = pointer to dogrec;
dogrec = record name: array 12 of char; age: integer; next: dogptr; end;
var q: dogptr; s: integer;
```

The following two statements form the body of a loop that sums the ages in a linked list of dogs.

```
s := s + q↑.age;
q := q↑.next
```

Show Keiko code for these two statements, omitting the run-time check that `q` is non-null.

```
LDGW s          ; Load the current value of s
LDGW q          ; Load the pointer q
LOADW          ; Dereference q to get the record
CONST <age_offset> ; Load the offset of the age field within the record
ADD            ; Calculate address of q^.age
LOADW          ; Load the value at q^.age
BINOP PLUS      ; Add q^.age to s
STGW s         ; Store the result back in s

; Move to the next element in the linked list
LDGW q          ; Load the pointer q
CONST <next_offset> ; Load the offset of the next field within the record
ADD            ; Calculate address of q^.next
LOADW          ; Load the value at q^.next
STGW q         ; Update q with q^.next
```

3.2: A small extension to the language of Lab 2 would be to allow blocks with local variables. We can extend the syntax by adding a new kind of statement:

```
stmt -> local decls in stmts end
```

For example, here is a program that prints 53:

```
var x, y: integer;
begin
    y := 4;
    local
        var y: integer;
    in y := 3 + 4;
        x := y * y
    end;
    print x + y
end.
```

As the example shows, variables in an inner block can have the same name as others in an outer block. Space for the local variables can be allocated statically, together with the space for global variables. Sketch the changes needed in our compiler to add this extension.

Answer:

Extend the Abstract Syntax Tree in `tree.mli` with the following:

```
type stmt =
  | Assign of string * expr
  | Print of expr
  | Local of (string * typ) list * stmt (* Represents local declarations
with a list of variable names and types, and a block of statements *)
  | Seq of stmt list
  | ...
```

We also need to add to the parser too:

```
stmt:
  ...
  | LOCAL decls IN stmts END { Local ($2, $4) }
  ...
```

As well as that, we also need to add to the `local` construct for this too.

```

let rec gen_stmt env stmt =
  match stmt with
  | Local (decls, stmts) ->
    let new_env = extend_env env decls in (* Extend the environment with
local variables *)
    let code = gen_stmt new_env stmts in
    release_locals decls; (* Release local variables after block ends *)
    code
  | ...

```

3.4: In some programming languages, it is a mistake to use the value of a variable if it has not first been initialised by assigning to it. Write a function that, for the language of Lab 1, tries to identify uses of variables that may be subject to this mistake. Discuss whether it is possible to do a perfect job, and if not, what sort of approximation to the truth it is best to make.

Answer: What we can do here is we can try and implement a static analysis function. This function would track any variables that are assigned and unassigned. It will then flag any variables that are trying to be used before assignment.

```

open Ast (* Assume this module defines the AST types *)

let find_uninitialized (program: ast) =
  let initialized = ref StringSet.empty in
  let warnings = ref [] in

  let rec check_stmt = function
    | Assign (var, _) ->
      initialized := StringSet.add var !initialized
    | Use var ->
      if not (StringSet.mem var !initialized) then
        warnings := ("Uninitialized use of " ^ var) :: !warnings
    | If (cond, then_branch, else_branch) ->
      check_expr cond;
      check_stmt then_branch;
      check_stmt else_branch
    | While (cond, body) ->
      check_expr cond;
      check_stmt body
    | _ -> ()

  and check_expr = function
    | Var var -> check_stmt (Use var)

```

```
| _ -> ()
```

```
in
```

```
List.iter check_stmt program;
```

```
!warnings
```